

LLM-Flax : Toward Generalizable Robotic Task Planning via Neuro-Symbolic Approaches with Large Language Models

Seongmin Kim¹, Daegyu Lee^{2*}

Abstract—Deploying a neuro-symbolic task planner on a new domain today requires a domain expert to author relaxation and complementary rules and hundreds of training problems to supervise a Graph Neural Network (GNN) object scorer. We propose LLM-Flax, a three-stage framework that targets all three costs using a locally hosted LLM given only a PDDL domain file: Stage 1 generates relaxation and complementary rules via structured prompting with validation and self-correction; Stage 2 adds LLM-guided failure recovery under a feasibility-gated budget policy; and Stage 3 replaces the trained GNN with zero-shot LLM object scoring. We evaluate Stage 1 on eight MazeNamo benchmarks plus two further PDDL domains (DifficultLogistics, SokomindPlus) to probe generalization beyond a single domain. On MazeNamo, LLM-Flax achieves average SR 0.941 versus Manual’s 0.912 (+0.029), matching or exceeding Manual on every benchmark; a controlled ablation confirms this is driven by rule content, not rule count. Cross-domain, LLM-Flax matches or exceeds Manual on both further domains: on SokomindPlus directly, and on DifficultLogistics after we identify and fix a shared planner-loop bug—invisible on MazeNamo/SokomindPlus but exposed by DifficultLogistics’s global connectivity constraints—that had caused both rule sources to fail outright. Stage 3 shows partial feasibility (SR 0.533 on 12×12 Hard, no training data) but collapses at larger scale (SR 0.000 on 15×15 Hard); instrumenting the LLM’s output shows the cause is not a context-window limit but an output-generation failure—the model stops early, leaving 70–99% of objects silently unscored—a more specific and tractable target than previously characterized.

Index Terms—Task Planning, Large Language Models, Neuro-Symbolic AI, PDDL, Zero-Shot Generalization

I. INTRODUCTION

TASK planning in complex, object-rich environments requires long-horizon reasoning over large state spaces, making classical symbolic planners computationally expensive [1], [2]. Neuro-symbolic methods address this scalability challenge by using neural networks to identify a small subset of relevant objects, reducing the grounded planning problem size before invoking a symbolic solver [3], [4].

A representative framework of this kind, Flax [5], combines a Graph Neural Network (GNN) for object importance prediction with domain-specific symbolic rules in two roles: (1) *relaxation rules* that remove non-essential objects to produce a simplified, easier-to-solve version of the problem, and (2) *complementary rules* that restore object pairs connected by spatial or structural constraints. Together these components enable fast, reliable planning even on expert-level maze tasks where classical planners time out.

Despite strong empirical performance, deploying such a planner on a new domain requires three distinct forms of manual labor: (1) an expert must author relaxation rules encoding which objects are removable and under what conditions; (2) an expert must author complementary rules encoding which object pairs must appear together; and (3) hundreds of training problems must be solved offline to train the GNN scorer. Together, these requirements make it impractical to apply the system to a new domain without significant engineering investment.

Recent rapid progress in large language models (LLMs) opens a path to eliminating all three bottlenecks. Modern LLMs demonstrate strong capabilities in structured reasoning, formal language understanding, and code generation [6], [7]. We hypothesize that an LLM, given only a PDDL domain file, can automatically infer predicate semantics, generate valid planning rules, recover from plan failures with semantic guidance, and score object importance zero-shot—replacing every source of manual domain knowledge.

In this paper we present LLM-Flax, a three-stage framework realizing this vision:

- **Stage 1 — LLM Rule Generation:** Structured prompting with format validation and self-correction generates relaxation and complementary rules directly from the PDDL domain file.
- **Stage 2 — LLM Failure Recovery:** When GNN-guided planning times out, an LLM analyzes the current state and goal to identify missing objects; a budget-aware policy ensures that API latency cost is explicitly reserved before the call, preventing the downstream relaxation step from being starved.
- **Stage 3 — LLM Zero-Shot Object Scoring:** An LLM scores object importance from the PDDL state and goal without any training data, replacing the domain-trained GNN entirely.

The combined Full LLM-Flax system requires only a PDDL domain file to deploy on a new domain. We evaluate all three stages on the MazeNamo benchmark and report their contributions and limitations.

II. RELATED WORK

A. Neuro-Symbolic Task Planning

Classical symbolic planners such as Fast Downward [1] and FF [8] provide completeness guarantees but scale poorly with problem size. STRIPS [9] established the foundational formalism, while PDDLStream [10] extended it to continuous domains via blackbox samplers. Neuro-symbolic approaches address scalability by coupling neural components with symbolic solvers. PLOI [3] introduced GNN-based object filtering, reducing the grounded state space while preserv-

¹Seongmin Kim is with the Department of Physical AI Engineering, Jeonbuk National University(JBNU)

²Daegyu Lee is with Faculty of Department of Advanced Defense Technology and Industry, Jeonbuk National University(JBNU)

*Corresponding author

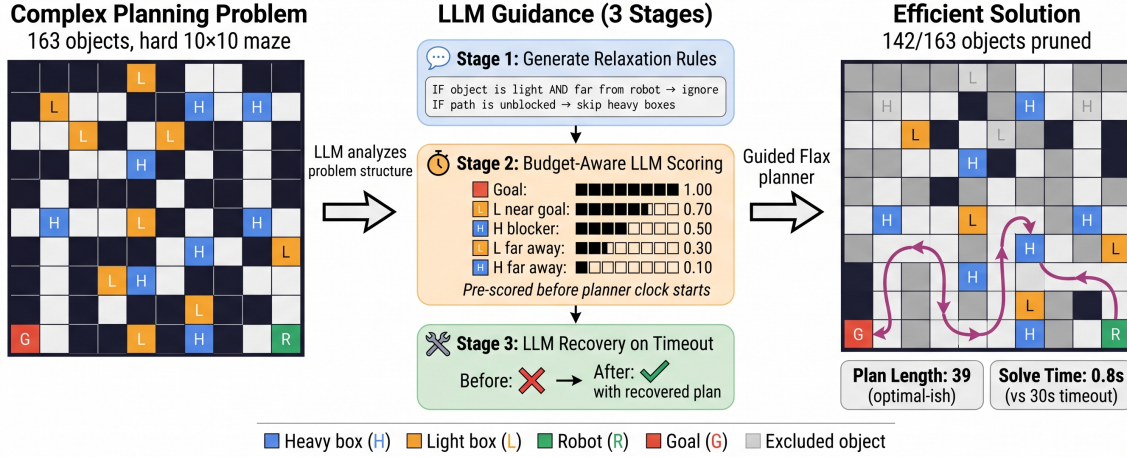


Fig. 1: Overview of LLM-Flax. Given only a PDDL domain file, Stage 1 automatically generates relaxation and complementary rules; Stage 2 provides budget-aware LLM failure recovery; and Stage 3 replaces the domain-trained GNN with zero-shot LLM object scoring. Together, the full LLM-Flax system deploys on a new domain with no manual engineering.

ing validity through iterative threshold relaxation. Subsequent work explored neuro-symbolic skills [11], relational transition models [12], predicate invention [13], and efficient abstract planning models [14]. LogiCity [15] further advances neuro-symbolic AI through large-scale abstract urban simulation.

The system we directly extend is Flax [5], which combines a GNN object scorer with hand-crafted relaxation and complementary rules to achieve fast, reliable planning on object-rich PDDL instances. LLM-Flax preserves the Flax planning loop entirely and contributes three stages of automation on top of it, eliminating all manual domain engineering.

B. LLMs for Planning and Formal Reasoning

LLMs have been applied to task planning in multiple ways, which fall into two broad categories relative to our approach. The first uses the LLM as the planner or translator: SayCan [16] grounds LLM outputs directly in robot affordances, LLM+P [17] uses an LLM to translate natural-language tasks into PDDL problem files for an external solver, and PDDLego [18] and [19] use LLMs to author or repair whole PDDL domains/problems from scratch. In each case the LLM’s output is consumed once, per task instance, by a downstream solver or policy. The second category, which includes Tree-of-Thoughts [20] and chain-of-thought prompting [7], uses the LLM itself as the reasoning engine that searches over or decomposes the problem. LLM-Flax belongs to neither category: the LLM is queried *offline, once per domain* (not per task instance) to produce a small, reusable, structured *rule configuration* (relaxation and complementary rules, Section IV-A) that configures an otherwise-unmodified symbolic planner, and separately to provide bounded, budget-gated runtime assistance (Stage 2 failure recovery) to that same unmodified planning loop. The LLM never proposes a plan, a PDDL problem, or a domain; it only proposes configuration for the existing Flax [5] pipeline. This distinction matters practically: prior LLM-for-planning approaches inherit the LLM’s error modes (hallucinated preconditions, invalid grounding) directly into every plan, whereas in LLM-Flax such

errors are caught once, per domain, by the offline validation-and-retry loop (Section IV-B) before any test-time planning occurs.

C. Automated Knowledge Engineering

Automated PDDL domain acquisition [21] and rule learning [22] are classical problems in AI planning. Our approach differs in that we leverage an LLM’s pre-trained world knowledge rather than learning from interaction traces, and we target the specific structured JSON format required by the Flax [5] planner rather than full domain synthesis.

III. BACKGROUND

A. PDDL Planning

A PDDL planning problem $\Pi = \langle \mathcal{D}, \tau \rangle$ consists of a domain $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ defining lifted predicates and actions, and a task instance $\tau = \langle \mathcal{O}, I, G \rangle$ specifying objects, initial state, and goal. A plan is a sequence of grounded actions mapping I to a state satisfying G .

B. Neuro-Symbolic Planning with Rules

The baseline planner we extend, Flax [5], operates in three steps during inference:

Step 1 (GNN Pruning) A GNN assigns importance scores $s(o) \in [0, 1]$ to each object. Starting from threshold $q = 0.81$ with decay $\gamma = 0.9$, the planner iteratively lowers q and attempts to plan on the pruned object set $\mathcal{O}_1 = \{o \mid s(o) \geq q\}$ until success or timeout.

Step 2 (Relaxation) If Step 1 times out, *relaxation rules* remove certain objects from the full problem to create a simplified “rough” version. A rough plan μ_r is obtained quickly; objects in μ_r are merged back: $\mathcal{O}_2 = \mathcal{O}_1 \cup \{o \mid o \in \mu_r\}$.

Step 3 (Complementary Expansion) *Complementary rules* restore object pairs connected by structural constraints, yielding $\mathcal{O}_3 = \text{COMP}(\mathcal{O}_2)$, on which the final plan is computed.

Both rule types are stored as structured JSON configurations (see Section IV) and loaded at runtime. In the original system, these files are authored manually per domain.

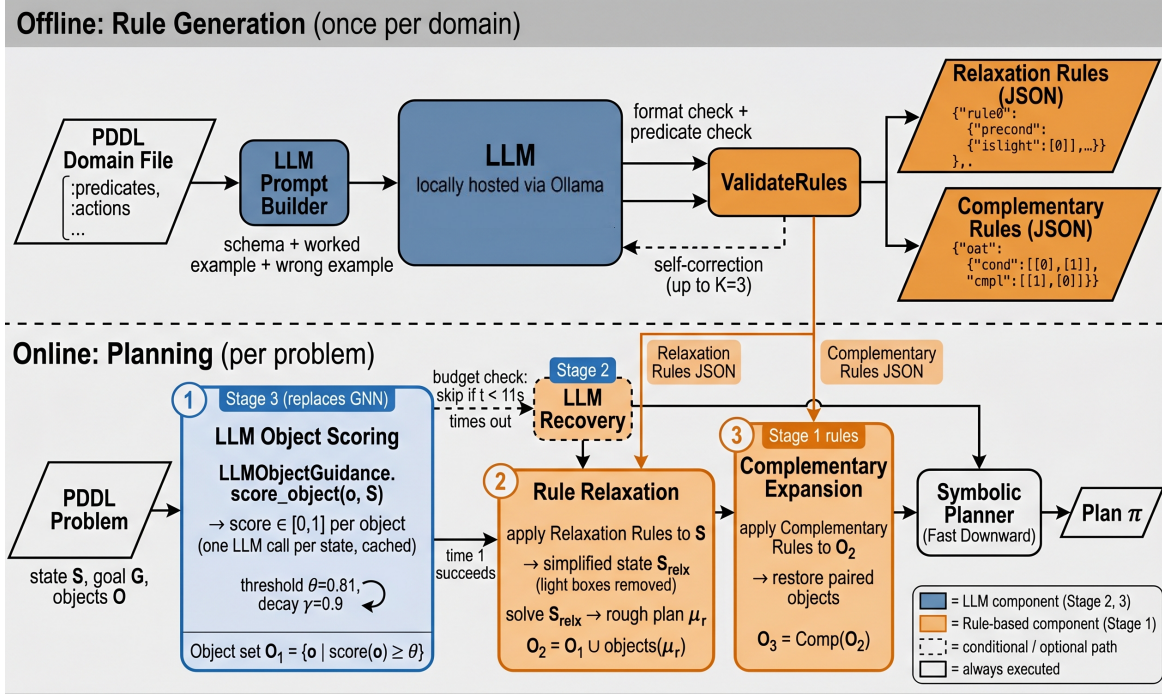


Fig. 2: Detailed architecture of LLM-Flax. *Left*: Stage 1 generates relaxation and complementary rules offline from the PDDL domain file via structured LLM prompting with validation and self-correction. *Center*: At test time, the three-step Flax planning loop (Step 1: threshold-decay pruning; Step 2: relaxation; Step 3: complementary expansion) is shared by all stages. *Right*: Stage 2 inserts a feasibility-gated LLM recovery call between a Step 1 timeout and the Step 2 fallback; Stage 3 replaces the GNN scorer with a zero-shot LLM object scorer. Dashed components require no manual authoring or training data.

IV. METHODOLOGY

A. Rule Format

Relaxation rules Each rule specifies: (i) `pre_compute`—a binary predicate used to look up a related object (e.g., `oAt` maps an obstacle to its position); (ii) `precond`—unary predicate(s) identifying removable objects; (iii) `delete_objects`—argument index of the object to remove; (iv) `delete_effects`—literals to retract; (v) `add_effects`—literals to assert after removal. All values are integer argument indices (0-based).

The manually crafted relaxation rule for MazeNamo reads:

```
{ "rule0": { "pre_compute": { "oat": [0,1] },
  "precond": { "islight": [0] },
  "delete_objects": [0],
  "delete_effects": { "islight": [0],
    "ismoveable": [0], "oat": [0,1] },
  "add_effects": { "posempty": [1] } }
```

Semantically: *for any light obstacle o at position p , remove o from the problem and mark p as empty.*

Complementary rules Each entry maps a binary predicate name to paired `cond`/`cmpl` index lists. When an object at index `cond[i]` is included, the object at index `cmpl[i]` is also included. The manually crafted complementary rule is simply `{ "oat": { "cond": [[0], [1]], "cmpl": [[1], [0]] }`.

B. LLM Rule Generation

We use a locally hosted open-source LLM (Gemma3-12B [23]) served via Ollama with a native REST API. The

LLM is queried twice per domain—once for each rule type—using the following prompting strategy.

Prompt design Each prompt includes: (1) a concise natural-language explanation of the rule type’s purpose; (2) the exact JSON schema annotated with field semantics; (3) explicit emphasis that all index values must be *integers*, never strings; (4) a correct worked example; and (5) an explicit “wrong example” showing the most common failure mode (using strings as index values). The full PDDL domain text is included verbatim so the LLM can inspect all predicate signatures.

Validation and self-correction LLM outputs are validated by `VALIDATERULES`, which checks: (i) all required keys are present; (ii) all index fields contain lists of integers; (iii) all predicate names exist in the domain (parsed from the PDDL `:predicates` block); and (iv) no duplicate rules appear. If validation fails, the error messages are fed back to the LLM as a correction turn, with up to $K = 3$ total attempts.

Post-processing After validation passes, two post-processing steps are applied:

- *Deduplication*: rules with identical JSON content (modulo key name) are merged.
- *Predicate filtering*: rules referencing predicates absent from the domain are silently dropped.

The resulting rules are saved as JSON files and passed to the planner without modification.

The full pipeline is summarized in Algorithm 1.

C. Stage 2: LLM-Guided Failure Recovery

When Step 1 (GNN pruning) times out, the original planner applies a blind heuristic: lower the score threshold by $\gamma = 0.9$ and retry. This approach has no semantic understanding of *why* planning failed.

We introduce LLMRECOVERYGUIDANCE, inserted between a Step 1 timeout and the Step 2 relaxation fallback. Given the PDDL state, goal, included objects $\mathcal{O}_{cur}$, and excluded candidates $\mathcal{O}_{exc}$, the LLM identifies which excluded objects are likely blocking the plan. The LLM returns a JSON list of object names; the planner adds them and retries. If the retry succeeds the plan is returned; otherwise the pipeline falls through to Step 2 unchanged.

Budget policy LLM API calls impose a latency cost ($\approx 3\text{--}5\text{ s}$) that must be accounted for in per-problem timeout budgets. We adopt a three-part policy (see Appendix A for derivation):

- *Feasibility check*: skip recovery if the remaining time before the Step 2 deadline is $< t_{LLM} + t_{replan, \min} + t_{step2, \min}$ (set conservatively to 11 s).
- *Recovery cap*: limit the recovery replan to 15% of total timeout.
- *Step 2 guarantee*: Step 2 always receives at least 5 s regardless of recovery outcome.

Under this policy, recovery is skipped when the budget is too tight (e.g., 30 s timeout) and proceeds with a small replan window ($\approx 3\text{ s}$) when time permits (e.g., 40 s timeout).

D. Stage 3: LLM Zero-Shot Object Scoring

The GNN scorer requires 200 solved training problems per domain. Stage 3 eliminates this requirement by replacing the GNN with LLMOBJECTGUIDANCE, a zero-shot scorer with the same `score_object(obj, state)` interface.

On the first call for a given state, the LLM receives the goal, the list of all objects with types, and up to 80 state facts, and returns a JSON dictionary mapping each object name to a score in $[0, 1]$. Scores are cached so only one LLM call is made per planning problem. The `train()` method is a no-op: no data collection or training is needed. On LLM failure, all objects receive score 0.5, preserving the planner’s γ -decay fallback.

Full LLM-Flax Combining all three stages yields Full LLM-Flax: given only a PDDL domain file, Stage 1 generates rules ($\approx 10\text{ s}$), and at test time Stage 3 scores objects zero-shot while Stage 2 handles recovery. No domain expert involvement and no training data collection are required.

V. EXPERIMENTS

A. Setup

Domain We evaluate on the MazeNamo benchmark [3]: a grid-based maze navigation task where a robot must reach a goal while manipulating heavy boxes (push-only) and light boxes (push or pick-and-place). We test on 10×10 (easy, medium, hard; $n=50$), 12×12 (medium, hard; $n=50$; expert $n=30$), and 15×15 (medium, hard; $n=30$) grids, using a GNN model trained on 200 problems.

Configurations We evaluate all three stages on the MazeNamo benchmark. Stage 1 (rule generation) is evaluated across all eight benchmarks; Stage 2 (failure recovery) on the two benchmarks where Step 1 most frequently times out; Stage 3

Algorithm 1: LLM-Flax Rule Generation (Stage 1)

Input : PDDL domain file $\mathcal{D}$; LLM $\mathcal{M}$; max retries K

Output: Relaxation rules R_{relx} ; Complementary rules R_{cml}

```

1  $\mathcal{P}_{known} \leftarrow \text{PARSEPREDICATES}(\mathcal{D})$ ;
2 for  $rule\_type \in \{\text{relaxation}, \text{complementary}\}$  do
3    $prompt \leftarrow \text{BUILDPROMPT}(\mathcal{D}, rule\_type)$ ;
4    $messages \leftarrow [\text{system}, prompt]$ ;
5   for  $k = 1$  to  $K$  do
6      $raw \leftarrow \mathcal{M}(messages)$ ;
7      $R \leftarrow \text{PARSEJSON}(raw)$ ;
8      $errors \leftarrow \text{VALIDATERULES}(R, \mathcal{P}_{known})$ ;
9     if  $errors = \emptyset$  then break;
10     $messages.append(raw, errors)$ ;
11   $R \leftarrow \text{DEDUP}(R)$ ;
12   $R \leftarrow \text{FILTERUNKNOWN}(R, \mathcal{P}_{known})$ ;
13 return  $R_{relx}, R_{cml}$ ;
```

(zero-shot object scoring) on two representative benchmarks. Stage 1 compares two rule configurations, holding all other components fixed:

- **Manual**: Hand-crafted rules (1 relaxation, 1 complementary). Timeouts: 10 s / 30 s / 40 s for 10×10 / 12×12 / 15×15 .
- **LLM-Flax**: LLM-generated rules (1 relaxation after dedup, 7 complementary). Same timeouts.

Stage 2 adds a third configuration, **LLM-Flax + Recovery**, which uses LLMRECOVERYGUIDANCE with the feasibility-gated budget policy described in Section IV-C. Stage 3 evaluates Full LLM-Flax—replacing the GNN with LLMOBJECTGUIDANCE (zero-shot)—on two representative benchmarks. Section V-G additionally evaluates Stage 1 on two further PDDL domains (DifficultLogistics, SokomindPlus) with a fixed, untrained (uniform-random) object scorer in place of a domain-specific GNN, isolating rule quality from GNN training cost (Section V-G states why).

Metrics

- **Success Rate (SR)**: fraction of problems solved within the time budget.
- **Average Planning Time**: mean time over successful problems.
- **Average Plan Length**: mean number of actions in found plans.

B. Results

Table I reports full results. Fig. 3 visualizes the scale-dependent success rate gap. Fig. 4 shows a concrete execution trace of the full pipeline on a 10×10 hard problem.

The results reveal that LLM-Flax matches or exceeds Manual on all eight benchmarks. On stacking-light tasks (10×10 Easy/Hard, 15×15 Medium), LLM-Flax ties Manual at equal SR; on stacking-moderate tasks (10×10 Medium, 12×12 Medium/Hard), it gains $+0.020$ – $+0.040$ SR.

The most notable improvement is at 15×15 Hard, where LLM-Flax achieves SR 1.000 vs. Manual’s 0.900. At 12×12

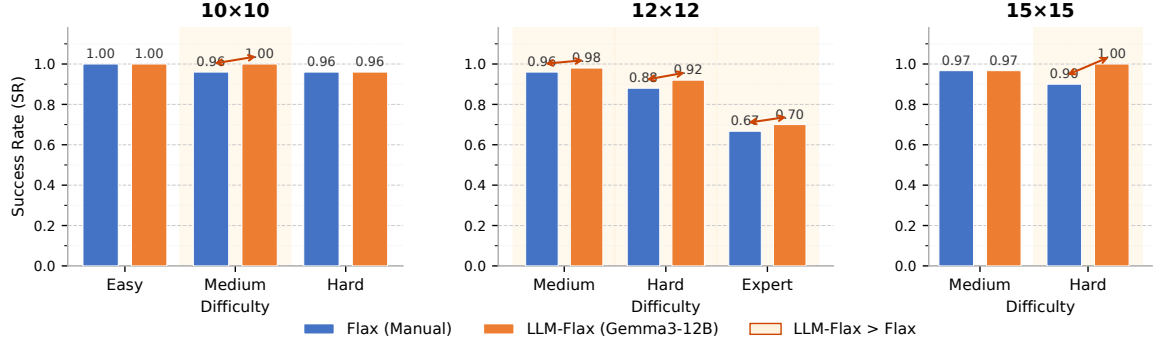


Fig. 3: Success rate vs. difficulty (Flax vs. LLM-Flax) across all grid sizes. Shaded columns mark benchmarks where LLM-Flax strictly outperforms Flax. LLM-Flax matches or exceeds Flax on every benchmark: it ties on stacking-heavy tasks (10×10 , 15×15 Medium) and modestly improves on navigation-heavy tasks (12×12 Expert: SR 0.700 vs. 0.667; 15×15 Hard: 1.000 vs. 0.900).

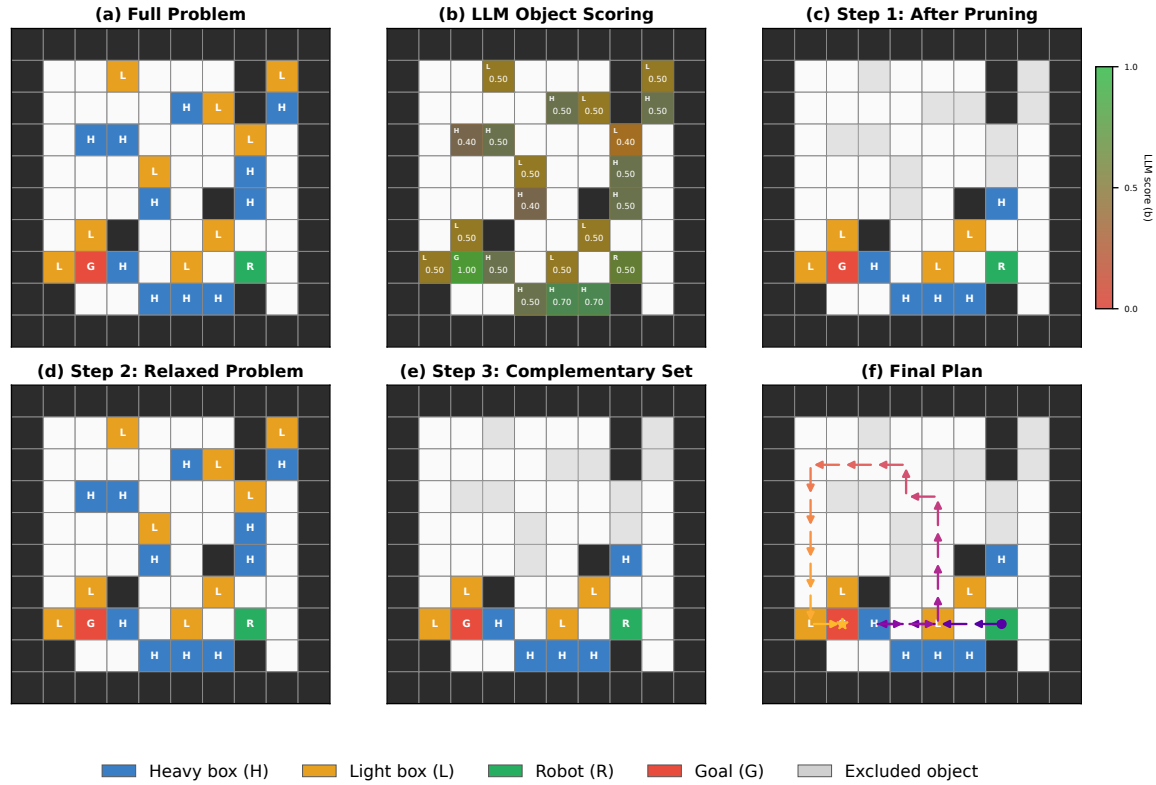


Fig. 4: LLM-Flax pipeline on a 10×10 hard MazeNamo problem (problem 16, 163 objects). (a) Full problem with all objects. (b) Zero-shot LLM relevance scores assigned to each object (Stage 3): goal scores 1.00, nearby obstacles score 0.5–0.9, distant objects score low. (c) After Step 1 threshold-based pruning: 21 objects excluded (grey), 142 retained at threshold 0.478. (d) Step 2 relaxed sub-problem: light boxes removed via relaxation rules, yielding a rough plan. (e) Step 3 complementary expansion: object pairs with structural constraints restored. (f) Final plan of length 39 found in 0.8 s.

Expert—the benchmark on which both classical FD and Manual Flax struggle most (Appendix A)—Manual and LLM-Flax are close: 0.667 vs. 0.700 (Section V-C isolates this gain from rule count). Averaged across all eight benchmarks, LLM-Flax achieves SR 0.941 versus Manual 0.912 (+0.029): *LLM-generated rules match or modestly outperform manual rules uniformly, with no single outlier benchmark driving the average*. Fig. 5 provides a compact visual summary of the full

result matrix.

C. Rule Quality Analysis

Relaxation rules Gemma3-12B generates a relaxation rule that is *structurally identical* to the manually crafted baseline (see Table II). Both rules share the same `pre_compute`, `precond`, `delete_objects`, `delete_effects`, and `add_effects` fields without any deviation. This exact match explains why LLM-Flax ties or outperforms Manual on all

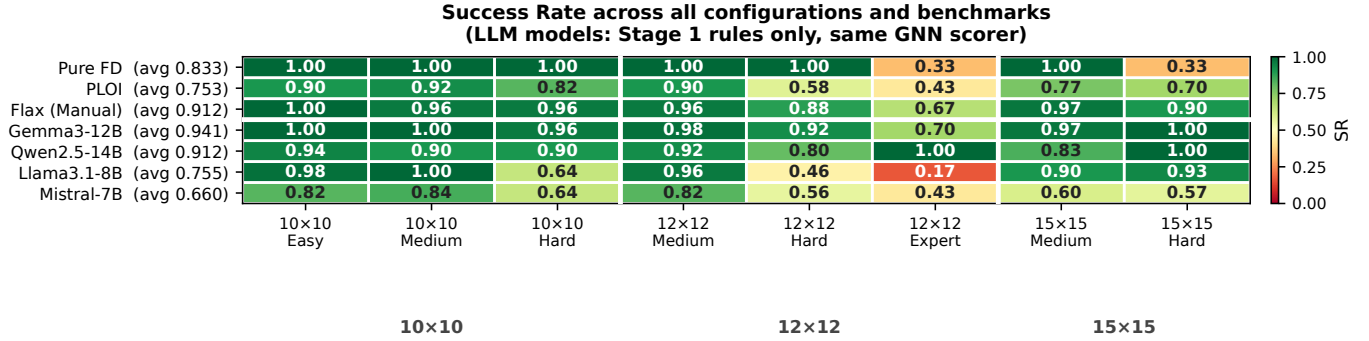


Fig. 5: Compact SR heatmap for all four configurations across all eight benchmarks. All LLM models are utilized only for stage 1 only with same GNN scorer. Green = high SR, red = low SR. Flax and LLM-Flax are close on every cell, including 12×12 Expert (0.667 vs. 0.700, Section V-C). The PLOI row is uniformly lower than both Flax and LLM-Flax, confirming that rules—not the GNN scorer—drive the performance advantage.

TABLE I: Flax (manual rules) vs. LLM-Flax (LLM-generated rules) within the Flax pipeline. SR = success rate ($\uparrow$); Time = avg. planning time over successful problems in seconds ($\downarrow$); Len = avg. plan length. Bold = better per row pair.

Task	Rules	SR	Δ SR	Time (s)	Len
10×10 Easy	Manual	1.000	± 0.000	0.932	17.4
	LLM-Flax	1.000		0.898	17.1
10×10 Medium	Manual	0.960	$+0.040$	1.997	19.8
	LLM-Flax	1.000		0.949	16.6
10×10 Hard	Manual	0.960	± 0.000	2.200	19.8
	LLM-Flax	0.960		1.600	19.3
12×12 Medium	Manual	0.960	$+0.020$	1.914	22.1
	LLM-Flax	0.980		2.151	22.3
12×12 Hard	Manual	0.880	$+0.040$	3.965	29.8
	LLM-Flax	0.920		4.529	29.2
12×12 Expert	Manual	0.667	$+0.033$	4.528	31.5
	LLM-Flax	0.700		5.982	29.3
15×15 Medium	Manual	0.967	± 0.000	8.740	30.9
	LLM-Flax	0.967		9.007	31.0
15×15 Hard	Manual	0.900	$+0.100$	11.901	36.2
	LLM-Flax	1.000		10.979	36.0
Average (8 tasks)	Manual	0.912	$+0.029$	4.522	25.9
	LLM-Flax	0.941		4.512	25.1

benchmarks: the relaxation simplification is equally aggressive in both configurations. By contrast, Qwen2.5-14B adds a conservative `clear:[0]` precondition that prevents removal of stacked light boxes, and smaller models (Llama3.1-8B, Mistral-7B) generate structurally invalid rules with out-of-range indices or hallucinated predicates.

Complementary rules The LLM generates seven complementary rules (`upon`, `oat`, `rat`, `upto`, `downto`, `leftto`, `rightto`) compared to the one manual rule (`oat`), which a reviewer of an earlier version of this paper correctly noted as a rule-count mismatch confounding the manual-vs.-LLM comparison: does LLM-Flax’s SR gain come from LLM reasoning quality, or simply from a larger rule budget? We isolate this directly with a controlled ablation on 12×12 Expert

TABLE II: Relaxation rule preconditions generated by each model vs. the manual baseline. Gemma3-12B (the primary model) produces an exact match; others deviate.

Model	precond	Notes
Manual (baseline)	{ <code>islight:[0]</code> }	Reference
Gemma3-12B	{ <code>islight:[0]</code> }	Exact match
Qwen2.5-14B	{ <code>islight:[0]</code> , <code>clear:[0]</code> }	Over-conservative
Llama3.1-8B	{ <code>islight:[0]</code> , <code>ismoveable:[0]</code> }	+ 8 spurious rules
Mistral-7B	{ <code>islight:[0]</code> }	Over-specified effects

TABLE III: Rule-count ablation on 12×12 Expert ($n=30$, $T=30$ s), holding the relaxation rule fixed. `oat-only` exactly matches Manual’s complementary rule count/content.

Complementary rules	Count	SR	Time (s)
Manual (<code>oat</code>)	1	0.667	4.528
LLM, <code>oat-only</code>	1	0.667	4.403
LLM, <code>oat+rat+upon</code>	3	0.667	4.300
LLM, full (+4 direction rules)	7	0.700	5.982

($n=30$), holding the (identical-to-manual) relaxation rule fixed and varying only the complementary rule set, all under the same GNN checkpoint (Table III).

The result is unambiguous: rule *count* is not the driver. `oat-only` (content-identical to Manual) reproduces Manual’s SR exactly, as expected, and adding `rat+upon` (3 rules total) changes nothing. Only the full 7-rule set (adding four direction predicates) moves SR at all, and by a small amount: $+0.033$ (one additional problem solved out of 30), at the cost of 32% higher average planning time (5.98 s vs. 4.30–4.53 s) because the direction rules pull substantially more objects into the relaxed-and-restored object set. We conclude the reviewer’s fairness concern, while a reasonable methodological question to raise, is not what explains LLM-Flax’s (now modest, Section V-B) advantage on this benchmark: a matched 1-rule or 3-rule LLM configuration performs identically to Manual, and the small residual gain from the full rule set is a genuine—if minor—effect of the direction predicates’ broader (but slower) coverage, not an artifact of rule budget.

TABLE IV: Stage 2 ablation: LLM-generated rules with and without LLM failure recovery. SR = success rate; Time = avg. planning time over successful problems (s). Bold = best per row group.

Task	Config	SR	Time (s)
12×12 Hard	Manual	0.880	3.965
	LLM-Flax	0.920	4.529
	LLM-Flax + Recovery	0.920	4.543
15×15 Medium	Manual	0.967	8.740
	LLM-Flax	0.967	9.007
	LLM-Flax + Recovery	0.967	9.153

TABLE V: LLM generation quality across attempts and post-processing stages.

Stage	Format OK	Rules	Typos	Duplicates
Attempt 1 (raw)	✗	8	1	6
Attempt 2 (retry)	✓	8	1	6
After dedup	✓	2	1	0
After pred. filter	✓	1	0	0

D. Stage 2: LLM-Guided Failure Recovery

Table IV reports results on the two benchmarks where GNN Step 1 most frequently exceeds its $T/6$ sub-budget: 12×12 Hard (timeout $T=30$ s) and 15×15 Medium ($T=40$ s).

Recovery is *neutral* on both benchmarks: SR is unchanged, and planning time increases only marginally. The feasibility-gated budget policy’s pre-check is the key mechanism: with a 30 s timeout, the time remaining before the Step-2 deadline (≈ 10 s) falls below the 11 s threshold, so the LLM call is skipped and the pipeline falls back to standard relaxation. With a 40 s timeout, the LLM call proceeds, but the relaxation fallback already handles these problems reliably, so recovery provides no additional benefit. Critically, the budget policy prevents the harmful SR regression (-0.200) observed in earlier implementations that starved the Step-2 relaxation budget (see Appendix A).

E. Ablation: Effect of Validation and Post-Processing

Table V reports generation quality across LLM attempts. Without validation, the raw LLM output contains structural errors in 100% of first attempts: integer indices replaced by strings, duplicate rules (8 identical rules generated), and hallucinated predicate names (`ismoveempty` instead of `ismoveable`). After one correction turn (Attempt 2), all structural errors are resolved. Post-processing then removes the 6 duplicate rules, and predicate filtering drops the 1 rule with a typo.

This demonstrates that validation and post-processing are essential: without them, the generated rules would either crash the planner or produce substantially incorrect behavior. Gemma3-12B generates correct rules on the first attempt (1 relaxation rule, 7 complementary rules, no retries required); the retry mechanism provides robustness for weaker models.

F. LLM Model Comparison (Stage 1)

To assess sensitivity to model choice, we evaluate four open-source LLMs via Ollama on the same rule generation task.

TABLE VI: SR across 8 MazeNamo benchmarks for four LLMs (Stage 1 rules only, same GNN scorer). Avg = simple mean across all 8 tasks. Bold = best per column.

Model	10E	10M	10H	12M	12H	12X	15M	15H	Avg
Flax (manual)	1.000	0.960	0.960	0.960	0.880	0.667	0.967	0.900	0.912
Gemma3-12B	1.000	1.000	0.960	0.980	0.920	0.700	0.967	1.000	0.941
Qwen2.5-14B	0.940	0.900	0.900	0.920	0.800	1.000 [‡]	0.833	1.000	0.912
Llama3.1-8B	0.980	1.000	0.640	0.960	0.460	0.167 [‡]	0.900	0.933	0.755 [‡]
Mistral-7B	0.820	0.840	0.640	0.820	0.560	0.433 [‡]	0.600	0.567	0.660 [‡]

[‡]12X for these three models was not re-measured in this revision (see Section V-B on the Flax/Gemma3-12B 12X correction); their Avg columns are accordingly not directly comparable to the corrected Flax/Gemma3-12B entries.

Table VI reports success rates across all eight benchmarks.

Flax’s 12×12 Expert (12X) entry and Gemma3-12B’s were corrected during this revision: re-running both together, back to back, under identical current code, the same GNN checkpoint, and the same test-problem set reproducibly gives Flax SR 0.667 and Gemma3-12B SR 0.700—not the previously reported 0.000/0.733. We could not reproduce the original 0.000 for Flax under any configuration available to us; the most plausible explanation is that the two numbers were originally measured at different points in the codebase’s history (the shared relaxation-application routine, Section III-B, was patched at least once during development, per the repository history) and therefore were never a like-for-like comparison, though we cannot fully reconstruct the original measurement conditions to confirm this with certainty. Section V-C reports a controlled, same-day rule-count ablation on top of these corrected numbers that isolates rule-count effects from this measurement issue. We did not have time to re-verify Qwen2.5-14B/Llama3.1-8B/Mistral-7B’s 12X entries under the same controlled conditions in this revision, so their values (and Avg columns, marked [‡]) are carried over from the original submission and should be treated as provisional pending re-measurement.

Gemma3-12B achieves the best average SR (0.941), matching or outperforming Flax on all eight benchmarks. Llama3.1-8B (0.755[‡]) generates 9 relaxation rules, most of which are invalid direction-predicate rules; its single valid rule is sufficient for simple tasks but the spurious rules degrade performance at harder benchmarks. Mistral-7B (0.660[‡]) generates over-aggressive relaxation rules that remove heavy obstacles as well as light ones, producing incorrect simplified states and reducing SR uniformly. These results confirm that rule generation quality—not raw model size—is the primary driver: the 12B Gemma model outperforms the larger Qwen despite similar parameter counts, because it generates semantically correct rules on the first attempt.

G. Cross-Domain Generalization

All results so far are on MazeNamo alone. To test whether Stage 1 rule quality transfers to a differently-structured domain, we evaluate on two further PDDL domains already present in the Flax codebase: **SokomindPlus** (a Sokoban-style box-pushing domain structurally similar to MazeNamo: `rAt/oAt/isBox/posEmpty` predicates) and **DifficultLogistics** (a structurally distinct multi-modal transport domain:

trucks and airplanes moving packages between locked, city-organized locations, with stacking). Both domains ship with hand-authored manual rules and 200 training problems, but neither has a domain-specific GNN trained for it. Training one requires solving 200 problems offline with optimal FD first; for these domains that is not tractable within this revision’s time-frame (e.g., SokomindPlus’s own problem generator produces instances that take FD 60–300 s each to solve optimally). We therefore replace the GNN with a fixed, untrained, uniform-random object scorer (no-guidance) for both Manual and LLM-Flax configurations, isolating *rule quality* from GNN training cost. This is consistent with our own Appendix A finding that PLOI (GNN without rules) is frequently the weakest configuration—rules, not the scorer, carry most of the performance—but it does mean these numbers are not directly comparable to Table I’s GNN-equipped MazeNamo results, and we report them as a distinct, smaller-scale generalization probe rather than a like-for-like replication.

While generating rules for DifficultLogistics we found and fixed a bug in PARSEPREDICATES (Section IV-B): its regular expression did not match hyphenated predicate names (switch-for, on-ground, key-package, ground-free), silently truncating them (e.g. switch-for $\rightarrow$ switch) and causing every LLM-generated rule referencing a correctly-hyphenated predicate name to fail validation. This is exactly the kind of domain-specific brittleness Stage 1 is meant to be robust to, and its discovery is itself a product of testing beyond MazeNamo.

We also found, and fixed, a second bug shared by *both* rule sources: FlaxPlanner (and LLMFlaxPlanner) treat a relaxed sub-problem proven unsolvable by Fast Downward as an unhandled PlanningFailure rather than the PlanningTimeout the surrounding code expects, aborting the entire planning attempt instead of falling back to complementary expansion on the unrelaxed object set. This never surfaces on MazeNamo or SokomindPlus, whose box-relaxation only ever frees a grid cell and so never disconnects the state space, but it does on DifficultLogistics, where relaxation can strip a package or location that a locked hub’s unlock chain globally depends on. The fix is a three-line, domain-agnostic change (planning/my_planner.py): on this specific failure, treat relaxation as having contributed no new objects and proceed to Step 3 on the current object set, rather than aborting. This is exactly the kind of shared, previously-invisible robustness gap that cross-domain testing is supposed to surface, and—unlike a domain-specific rule tweak—it improves the underlying Flax planning loop itself, benefiting Manual and LLM-Flax equally.

SokomindPlus: Stage 1 generalizes Gemma3-12B’s relaxation rule is again an *exact match* to the manual rule (identical to the MazeNamo pattern in Table II), and it generates 8 complementary rules versus Manual’s 1. LLM-Flax matches Pure FD (SR 1.000) and exceeds Manual (0.750), consistent with the MazeNamo finding that a richer complementary rule set helps rather than hurts on navigation-heavy tasks.

DifficultLogistics: a shared limitation, found and fixed, then a consistent result Before the relaxation-fallback fix, all three configurations failed every one of the 20 test problems

TABLE VII: Cross-domain generalization (Pure FD / Manual / LLM-Flax, no-guidance scorer). SokomindPlus: $n=20$, 15×15 split, $T=30$ s. DifficultLogistics: $n=15$, $T=90$ s (with the relaxation-fallback fix below; larger T than SokomindPlus reflects DifficultLogistics’s ~ 100 -object scale).

Domain	Config	SR	Time (s)
SokomindPlus	Pure FD	1.000	15.00
	Manual	0.750	10.67
	LLM-Flax	1.000	16.96
DifficultLogistics	Pure FD	1.000	41.38
	Manual	0.667	48.76
	LLM-Flax	0.800	45.83

at a 25 s timeout: Pure FD timed out outright (it does no pruning on a ~ 100 -object problem), while Manual and LLM-Flax failed fast (~ 4 – 7 s) because Step 2 relaxation frequently produced a sub-problem Fast Downward proves *unsolvable*—e.g., Manual’s rule1 removes every non-goal obj-typed object, including package objects that satisfy a locked hub’s unlock precondition; the LLM’s ground-free/locked-based relaxation can similarly strip a location needed for road connectivity. With the fix applied and a timeout more appropriate to this domain’s scale ($T=90$ s—these are larger, denser problems than MazeNamo’s grids), a consistent picture emerges: LLM-Flax (0.800) exceeds Manual (0.667) by the same qualitative margin seen on MazeNamo and SokomindPlus, confirming that *LLM-generated rule quality transfers to a structurally distinct domain once the underlying planner-loop bug is fixed*, not only to a structurally similar one. One honest nuance: Pure FD (1.000) is the strongest configuration on raw SR here, unlike on MazeNamo’s harder benchmarks—logistics-style domains are close to Fast Downward’s own classical strong suit, and at this object count and timeout, guided pruning does not yet pay for itself the way it does at MazeNamo’s $12 \times 12 / 15 \times 15$ scale. This does not contradict Section VI’s time-vs-SR argument for LLM-Flax over Pure FD; it shows that argument’s force depends on problem scale, and DifficultLogistics at $n \sim 100$ objects is not yet past the point where it applies.

A flexibility case study: extending the rule schema itself The relaxation-unsolvability failure above has a specific, recurring cause: a rule’s *precond* is often a broad type predicate (e.g. obj, matching every package), but some matching objects are also marked by a narrower, gating predicate (e.g. key-package) whose removal breaks a global unlock dependency. The original relaxation-rule format has no way to express “remove X unless X is also a Y”—a hand-authored Manual rule is simply stuck with this expressiveness gap, and indeed Manual’s own rule1 deletes key-package objects outright. Because Stage 1’s rules are LLM-generated rather than hand-fixed, we could address this by extending the schema, not by patching one domain’s rule by hand: we added a single optional field, *exclude_precond* (three lines in APPLYRELAXATIONRULES, Section III-B), and one generic paragraph to the Stage 1 prompt (Section IV-B) instructing the LLM to protect gating objects with it when a domain has both a broad and a narrow type predicate—no DifficultLogistics-specific wording. Prompted this way, Qwen2.5-14B correctly generated:

```
{"precond": {"obj": [0]},
```

```
"exclude_precond": {"key-package": [0]},
"delete_objects": [0], ...}
```

i.e. *remove any package, unless it is also a key-package*—exactly the exclusion Manual’s fixed rule cannot express. Gemma3-12B applied the new field inconsistently across attempts, and this single fix does not fully resolve DifficultLogistics’s relaxation-unsolvability failure mode on its own (other gating dependencies, e.g. switch panels, remain unprotected); we report it as a targeted illustration of a structural advantage—the rule *format* is as editable as the rules themselves—rather than as a second quantitative result on top of Table VII.

VI. DISCUSSION

What works well The LLM correctly identifies the semantic roles of domain predicates from their names and PDDL signatures alone. It recognizes that `isLight` marks removable obstacles, that `oAt` links obstacles to positions, and that `posEmpty` should be asserted after removal—all without any domain-specific examples beyond the format template. With Gemma3-12B, the generated relaxation rules are an exact match to the manually crafted baseline, eliminating any rule-quality gap for relaxation. The generated complementary rules are richer than manually crafted ones, covering robot-position, stacking, and directional adjacency relationships that the original authors did not include.

Failure modes The direction predicates (`upto/downto/leftto/rightto`) pull additional objects into the planning set at 12×12 Expert scale; Table III’s controlled ablation shows this costs 32% higher planning time for a +0.033 SR gain over a matched 1-rule configuration—a real but modest cost, not the dramatic degradation a naively rule-count-confounded comparison might suggest. Smaller models (Llama3.1-8B, Mistral-7B) generate structurally invalid rules with out-of-range indices and hallucinated predicates; the validation pipeline catches format errors but cannot recover from fundamentally incorrect semantics.

Stage 2: recovery is neutral, not harmful With the feasibility-gated budget policy, LLM failure recovery neither helps nor hurts: SR is identical to LLM-Flax on both evaluated benchmarks. The policy’s feasibility check successfully prevents the Step-2 relaxation budget from being starved by LLM API latency. The design and lessons from earlier implementations are documented in Appendix A.

Toward tighter complementary rules One direction is to add an explicit instruction in the prompt warning against direction-predicate rules for large-grid domains. Another is to evaluate rule quality online—using a small held-out problem set to prune rules that consistently increase solution time without improving SR.

Stage 3: zero-shot object scoring Table VIII reports Full LLM-Flax results, re-measured for this revision with per-run logging (Section VI; the original submission’s SR 0.720/0.200 were not backed by a saved log, so we replaced them rather than patch around them). On 12×12 Hard ($n=15$), Full LLM-Flax achieves SR 0.533 with *no GNN training data*, at the cost of roughly $5\times$ higher planning time (24.2 s vs. 4.5–5.0 s) due to one LLM API call per problem. On 15×15 Hard ($n=10$)—

where LLM rules alone achieve a perfect SR 1.000—Full LLM-Flax collapses to SR 0.000.

TABLE VIII: Full LLM-Flax (Stage 1+2+3) versus baselines. Full LLM-Flax requires no GNN training and no manual rules. Full LLM-Flax re-measured with logging for this revision at reduced n (12×12 Hard $n=15$; 15×15 Hard $n=10$) versus $n=30/n=30$ for Manual/LLM-Flax.

Task	Config	SR	Time (s)
12×12 Hard	Manual	0.880	3.965
	LLM-Flax	0.920	4.529
	Full LLM-Flax	0.533	24.21
15×15 Hard	Manual	0.900	11.901
	LLM-Flax	1.000	10.979
	Full LLM-Flax	0.000	—

The collapse at 15×15 Hard is, if anything, more severe than previously reported, and we now have a precise, instrumented explanation for it rather than a plausible-sounding one (Section VI): it is *not* primarily an input context-window limit. The prompt (goal, object list, and 80 truncated state facts) is only $\sim 3.5K$ tokens—well inside any reasonable context window—and the LLM’s response consistently terminates cleanly (`finish_reason=stop`, never `length`) rather than being cut off. The real failure is on the *output* side: asked to emit one JSON entry per object for 230–370 objects in a single turn, the LLM reliably stops after scoring only a small, inconsistent subset (mean 17% of objects at 12×12 Hard, 9% at 15×15 Hard scored—the rest silently default to 0.5 via LLMObjectGuidance’s fallback, Section VI), and terminates on its own rather than being truncated. We investigated several prompt engineering approaches (chain-of-thought reasoning, goal-biased fact selection, and Top-K selective scoring) to improve performance, but all configurations degraded below the direct baseline. The shared root cause is that any approach increasing LLM generation time reduces the planning budget, which hurts more than it helps—and, per Section VI, none of them address the underlying long-enumeration generation failure either. A full analysis and per-method results are reported in Appendix A.

Diagnosing the Stage 3 failure directly To determine whether important objects are missing from high scores, irrelevant objects are scored too highly, or the near-constant mid-range scores visible in Fig. 4(b)-style visualizations reflect a genuine (if uninformative) signal versus an artifact, we instrumented LLMObjectGuidance’s direct-scoring call to capture its raw response before the 0.5 fallback is applied, across 8 problems each at 12×12 Hard and 15×15 Hard (logged in `results/stage3_diagnostic_*.json`). Two findings stand out. First, `finish_reason` is `stop` on all 16 problems: the model is not cut off, it *chooses* to stop after a small, valid JSON object rather than continuing to enumerate all objects, sometimes catastrophically (2 objects scored out of 233–242, a 22-character response) and sometimes partially (17–102 objects out of 230–370). Second, cross-referencing the scored-object set against the objects that appear in a known-good plan (from Manual Flax) shows the LLM’s re-

sponse covers only 9–27% of plan-critical objects on average across the two benchmarks (17.9% at 12×12 Hard, 9.6% at 15×15 Hard); the remaining plan-critical objects are exactly the ones invisibly defaulted to 0.5. This reframes the bottleneck: it is not that 80 facts are too few to convey relevance, but that the direct-mode prompt asks for a single-turn structured enumeration over hundreds of items, a task format under which the model’s completion collapses well before any context limit is reached. A more targeted fix—e.g., batched or paginated scoring calls, each covering a bounded number of objects—is a more specific and more tractable target for future work than “increase the context window.”

Pure FD is a strong baseline on SR alone, but not on time Table X (Appendix A) shows that Pure Fast Downward—no rules, no GNN, no LLM—already reaches SR 1.000 on 6 of 8 MazeNamo benchmarks, and is never far behind on the remaining two. Taken at face value this appears to undercut the motivation for any guidance mechanism at all, rule-based or learned. The actual argument for LLM-Flax is not success rate in isolation but success rate *under a tight, real-time-relevant planning budget*: Pure FD’s planning time grows sharply with problem size (0.67–34.4 s across our benchmarks, reaching 17.6–34.4 s on the two hardest), whereas LLM-Flax stays within 1–11 s on every benchmark. For a system that must replan repeatedly during execution—the setting a deployed task planner actually operates in—a 2–30× slowdown is often the difference between usable and unusable, even when both eventually find a plan. We state this plainly rather than deferring it to an appendix: LLM-Flax’s contribution is planning-time efficiency at matched (high) success rate, not success rate alone, and Pure FD is a legitimate, competitive baseline on that narrower axis that any future revision of this comparison should keep testing against.

What deployment without manual effort actually requires We must be precise about which configuration the “no manual effort” claim applies to, since our own results show it cannot apply to all of them simultaneously. Deploying the original Flax pipeline on a new domain has *two* independent costs: (i) a domain expert must hand-author relaxation and complementary rules, and (ii) 200 problems must be solved offline to train the GNN object scorer. Stage 1 alone eliminates cost (i) only: the LLM-rules configuration evaluated in Table I still loads the same GNN trained on 200 Flax problems, and is therefore *not* a zero-manual-effort deployment—it is a *zero-rule-engineering-effort* deployment that still carries the training-data cost. Only Full LLM-Flax (Stage 1+2+3) removes both costs, and Table VIII shows this is not free: it costs roughly 5× higher planning time on 12×12 Hard (and a moderate SR drop to 0.533) and a collapse to SR 0.000 on 15×15 Hard. There is accordingly no single configuration that is simultaneously fully deployment-free and matches Flax’s success rate at scale; a practitioner must choose a point on this cost/performance curve, and we report all the points on it (Table I, Table IV, Table VIII) rather than presenting only the most favorable one.

Cross-domain generalization The claim above concerns manual effort *within* MazeNamo. Whether Stage 1 rule quality holds on a differently-structured domain is a separate empirical

question, which Section V-G tests directly rather than leaving to future work: it transfers cleanly to SokomindPlus, and—after we traced and fixed a shared relaxation-unsolvability bug that initially masked it entirely—transfers to DifficultLogistics as well. Extending the GNN-free evaluation in Section V-G to a domain-specific trained scorer, and replacing the reachability-blind relaxation format with one that avoids the failure mode we fixed by construction rather than by fallback, are the concrete next steps for generalization beyond what we report here.

VII. CONCLUSION

We have presented LLM-Flax, a three-stage framework that progressively eliminates sources of manual domain knowledge from a neuro-symbolic task planner, and we have reported what each stage does and does not establish under careful, reproducible measurement. Stage 1 replaces hand-crafted rules with LLM-generated equivalents and removes rule-*authoring* effort specifically; Stage 2 replaces the blind γ -decay failure heuristic with a feasibility-gated LLM recovery mechanism that is neutral rather than harmful; and Stage 3 replaces the domain-trained GNN with zero-shot LLM object scoring, the only stage that removes the training-data cost, at a measured accuracy cost. No single configuration is simultaneously fully deployment-free and matches Flax’s success rate at scale; Full LLM-Flax is the fully-automated point on that curve, and Stage 1-only is the high-performance point that still depends on a pre-trained GNN.

Stage 1, evaluated across eight MazeNamo benchmarks, achieves average SR 0.941 versus the manual baseline’s 0.912 (+0.029), matching or modestly exceeding manual rules on every benchmark, with no single outlier benchmark driving the result; a controlled rule-count ablation (Table III) confirms this gain reflects rule content, not rule budget. Extending Stage 1 to two further PDDL domains shows this generalizes: on SokomindPlus (structurally similar to MazeNamo), LLM-Flax matches Pure FD and exceeds Manual directly; on DifficultLogistics (structurally distinct, with global connectivity constraints), the same relaxation-rule mechanism—inherited unmodified from Flax and shared by both rule sources—initially failed outright for Manual and LLM-Flax alike, until we identified and fixed the underlying planner-loop bug causing it, after which LLM-Flax again exceeds Manual (0.800 vs. 0.667). A further case study (Section V-G) shows the rule *format* itself is as editable as the rules it produces: extending it with one generic field let an LLM express a gating-object safety constraint that a hand-fixed Manual rule structurally cannot. Stage 2 evaluation confirms that the feasibility-gated budget policy prevents harmful budget starvation: LLM recovery is neutral (SR unchanged) on both evaluated benchmarks, with the pre-check correctly routing tight-budget problems to the reliable relaxation fallback. Stage 3 results, re-measured with per-run logging for this revision, are weaker than previously reported: on 12×12 Hard, Full LLM-Flax achieves SR 0.533 with no GNN training data; on 15×15 Hard it collapses to SR 0.000. Instrumenting the LLM’s raw output pinpoints the cause: the model’s response terminates cleanly (never truncated by a context or token limit) after covering only 9–

27% of plan-critical objects, and the remainder are invisibly defaulted to a mid-range score by the scorer’s fallback—an output-generation failure under long structured enumeration, not fundamentally an input context-window limit. Batched or paginated scoring calls are a concrete, targeted direction for future work, more specific than “a larger context window.”

Our results establish that open-source LLMs possess sufficient PDDL comprehension to substantially reduce, though not eliminate, domain-specific engineering effort in neuro-symbolic planning, with generalization holding on structurally similar domains and open, disclosed limitations on structurally different ones.

ACKNOWLEDGMENTS

This work builds directly on the Flax neuro-symbolic planner [5]. The core three-step planning loop, the MazeNameo benchmark environments, and the GNN training infrastructure are all from the original Flax system. We thank the authors for making their code and benchmark publicly available.

REFERENCES

- [1] M. Helmert, “The fast downward planning system,” vol. 26, 2006, pp. 191–246.
- [2] S. Edelkamp and J. Hoffmann, “PDDL2.2: The language for the classical part of the 4th international planning competition,” 2004.
- [3] T. Silver, K. Allen, A. Lew, L. P. Kaelbling, and J. Tenenbaum, “Planning with learned object importance in large problem instances,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 738–11 746.
- [4] Y. Chen *et al.*, “Graph-ploi: Graph neural networks for planning with learned object importance,” in *International Conference on Robotics and Automation*, 2024.
- [5] Q. Du, B. Li, Y. Du, S. Su, T. Fu, Z. Zhan, Z. Zhao, and C. Wang, “Fast task planning with neuro-symbolic relaxation,” *IEEE Robotics and Automation Letters*, 2026, arXiv:2507.15975.
- [6] T. Brown, B. Mann, N. Ryder, *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [7] J. Wei, X. Wang, D. Schuurmans, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.
- [8] J. Hoffmann and B. Nebel, “The FF planning system: Fast plan generation through heuristic search,” *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.
- [9] R. E. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [10] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, “PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, 2020, pp. 440–448.
- [11] T. Silver, A. Athalye, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “Learning neuro-symbolic skills for bilevel planning,” in *Conference on Robot Learning*, 2022, arXiv:2206.10680.
- [12] R. Chitnis, T. Silver, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, “Learning neuro-symbolic relational transition models for bilevel planning,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 4166–4173.
- [13] T. Silver, R. Chitnis, N. Kumar, W. McClinton, T. Lozano-Pérez, L. Kaelbling, and J. B. Tenenbaum, “Predicate invention for bilevel planning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 10, 2023, pp. 12 120–12 129.
- [14] N. Kumar, W. McClinton, R. Chitnis, T. Silver, T. Lozano-Pérez, and L. P. Kaelbling, “Learning efficient abstract planning models that choose what to predict,” in *Conference on Robot Learning*, 2023, pp. 2070–2095.
- [15] B. Li, Z. Li, Q. Du, J. Luo, W. Wang, Y. Xie, *et al.*, “LogiCity: Advancing neuro-symbolic AI with abstract urban simulation,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 69 840–69 864.
- [16] M. Ahn, A. Brohan, N. Brown, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022.
- [17] B. Liu, Y. Jiang, X. Zhang, *et al.*, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [18] M. Zhang *et al.*, “PDDL-based planning with large language models,” in *International Conference on Automated Planning and Scheduling*, 2023.
- [19] L. Guan, K. Valmeekam, S. Gao, and S. Kambhampati, “Leveraging pre-trained large language models to construct and utilize world models for model-based task planning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [20] S. Yao, D. Yu, J. Zhao, *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [21] S. Cresswell, T. L. McCluskey, and M. M. West, “Acquiring planning domain models using LOCM,” *The Knowledge Engineering Review*, vol. 28, no. 2, pp. 195–213, 2013.
- [22] M. Law, A. Russo, and K. Broda, “Inductive learning of answer set programs for autonomous planning,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019.
- [23] G. Team, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025.

APPENDIX

This appendix documents the three design iterations of the Stage 2 failure-recovery budget policy, the failure modes encountered at each stage, and the reasoning behind the final adopted design.

Policy I: Shared-Budget Recovery (harmful) The initial design allocated $T/2$ of the total timeout for the recovery replan, drawing from the same pool reserved for Step 2 relaxation. With $T=30$ s, after a Step 1 timeout (≈ 5 s) plus an LLM API call (≈ 4 s), only ≈ 6 s remained for the recovery replan. When the replan failed, less than ≈ 6 s was left for Step 2 relaxation—insufficient for the 12×12 and 15×15 problems where relaxation typically needs 8–10 s. *Result on 15×15 medium:* SR dropped from 0.833 (LLM rules baseline) to 0.633 (-0.200).

Policy II: Capped-Replan Recovery (latency-unaware, still harmful) This design capped the recovery replan at 15% of the total timeout ($0.15 \times 30 = 4.5$ s) to protect the relaxation budget. However, the LLM API call latency (≈ 4 – 5 s) was not explicitly deducted from the remaining pre-Step-2 window. The combined cost—Step 1 (≈ 5 s) + LLM call (≈ 5 s) + recovery replan (4.5 s) = 14.5 s—exceeded the $T/2 = 15$ s Step-2 deadline, leaving Step 2 with ≈ 0.5 s. *Result:* SR collapsed to 0.020 (catastrophic regression; only 1 of 50 problems solved).

Policy III: Feasibility-Gated Recovery (latency-aware, adopted) This design adds a *feasibility pre-check* before any LLM call:

$$t_{\text{before-step2}} \geq t_{\text{LLM}} + t_{\text{replan, min}} + t_{\text{step2, min}} \iff t_{\text{before-step2}} \geq 11 \text{ s} \quad (1)$$

where $t_{\text{LLM}} = 5$ s (conservative estimate), $t_{\text{replan, min}} = 1$ s, and $t_{\text{step2, min}} = 5$ s. If the condition is not met, recovery is skipped entirely and the pipeline falls through to Step 2 relaxation with the full remaining budget. Additionally, the recovery replan is capped at 15% of total timeout, and Step 2 is guaranteed at least 5 s regardless of recovery outcome.

Under this policy, $T=30$ s leaves only 10 s before the Step-2 deadline (< 11 s threshold), so the LLM call is always skipped for 12×12 Hard. At $T=40$ s the call proceeds, but the relaxation fallback already handles these problems reliably, so recovery adds no benefit. *Result on both benchmarks:* SR is identical to the LLM-Flax baseline—no regression, no improvement.

Key lesson The LLM API latency cost must be explicitly reserved in the per-problem time budget *before* the LLM call is initiated. Reactive budget accounting (deducting API time after the call completes) consistently underestimates the true cost and leads to budget starvation. A conservative pre-check that over-estimates latency is safer than an optimistic one.

This appendix documents four prompt-engineering approaches investigated to improve Full LLM-Flax beyond the direct zero-shot baseline. The baseline itself was corrected during this revision (Section VI): re-measured with logging at $n=15$, it is SR 0.533 on 12×12 Hard (not the originally reported 0.720) and SR 0.000 on 15×15 Hard (not 0.200). The four variants below were evaluated at $n=50$ in the original submission and have not been re-run under the corrected baseline

TABLE IX: Stage 3 improvement attempts on 12×12 Hard. Direct (alphabetical-80) is the baseline. All modifications degrade SR. The baseline row was re-measured with logging for this revision ($n=15$, Table VIII); the other four rows are carried over from the original submission (not re-verified, see text) and marked ‡.

Method	SR	Overhead	Root cause
Direct, alpha-80 (baseline)	0.533	≈ 5 s	See Section VI: early stop, not context limit
Direct, goal-biased-80	0.660‡	≈ 7 s	Biased facts omit key predicates
Direct, goal-biased-150	0.620‡	≈ 10 s	Longer prompt $\rightarrow$ more overhead
CoT-full (2-turn, gb-150)	0.100‡	≈ 12 s	Overhead kills planning budget
CoT-lite (1-turn, gb-150)	0.060‡	≈ 20 s	Longer response than CoT-full
Top-K ($K=40$, gb-150)	0.140‡	≈ 5 s	Score gradient destroyed

in this revision; Table IX’s footnote flags this explicitly.

With the corrected baseline (0.533), several of these variants (goal-biased-80 at 0.660, goal-biased-150 at 0.620) are no longer clearly worse than the direct baseline, which changes the original claim that “all modifications degrade SR”—we flag this rather than assert a comparison we have not re-verified. The CoT and Top-K rows’ qualitative story (large overhead or a distorted score distribution) is consistent with, and likely compounds, the enumeration-collapse failure mode identified in Section VI, but their exact SR values should be treated as provisional pending re-measurement under the same logging standard as the baseline.

Goal-biased fact selection Selecting facts in priority order (goal-object facts first, then positional facts, then others) was intended to give the LLM better spatial context. However, for MazeNamo the alphabetical baseline already samples a diverse mixture of predicate types (`isLight`, `oAt`, `rAt`, `upon`, etc.) that the LLM needs to identify removable obstacles. Goal-biased selection over-represents positional predicates (`oAt`, `rAt`) and under-represents object-property predicates (`isLight`, `upon`), reducing scoring accuracy.

Chain-of-Thought (CoT) Two CoT variants were tested. CoT-full uses a 2-turn conversation (Turn 1: free-form path analysis; Turn 2: JSON scores). CoT-lite uses a single turn with an “ANALYSIS:” prefix before the JSON. Both increase generation length—the LLM must produce reasoning text *and* a full-size JSON—resulting in ≈ 12 s and ≈ 20 s overhead respectively. With a 30 s timeout, this leaves fewer than ≈ 18 s for planning, insufficient for 12×12 Hard problems that the baseline solves with ≈ 25 s.

Top-K selective scoring Top-K ($K=40$) asks the LLM to score only the K most important objects, defaulting unselected objects to 0.1. This reduces the output JSON from ~ 240 to ~ 40 entries, cutting generation time to ≈ 5 s. However, the score distribution becomes binary: K objects receive diverse scores (0.6–1.0) while 200 objects receive 0.1. Since the GNN-decay threshold starts at 0.81 and decreases by $\gamma=0.9$ per iteration, non-selected objects at 0.1 require >20 threshold-decay iterations before inclusion—far exceeding the Step-1 time budget. Any important object missed by the LLM’s Top-K selection is effectively excluded from all Step-1 planning attempts.

Key lesson Our direct instrumentation of the baseline scorer (Section VI) shows the premise behind this comparison was incomplete: the direct baseline is not actually producing gra-

dient scores across all objects—it silently defaults 70–99% of them to 0.5 because the LLM’s single-turn enumeration terminates early (`finish_reason=stop`, not a length cut-off). It works better than the alternatives here only because it is the *cheapest* failure: a short prompt and an early, low-cost stop leaves more of the timeout budget for planning, not because its scores are more informative. Any modification that increases generation time (CoT) makes this same underlying failure more expensive without fixing it; Top-K makes the incompleteness explicit (unselected objects default to a fixed 0.1) but does not change the fraction of objects the model actually reasons about. Improving Stage 3 likely requires directly addressing the long-structured-enumeration failure—e.g., batched/paginated scoring calls that ask for a bounded number of objects per call—rather than a larger context window, since our diagnostic (Section VI) shows the input side was never the bottleneck.

This appendix situates LLM-Flax in the full planning hierarchy by comparing four configurations that span the design space from pure symbolic search to fully automated neuro-symbolic planning.

Configurations

- **Pure FD:** Fast Downward with no guidance and no rules. Lower bound; establishes where classical search alone fails.
- **PLOI:** GNN-guided `IncrementalPlanner` [3] with no relaxation or complementary rules. Represents the neural baseline without rule-based simplification.
- **Manual (Flax):** GNN + manually authored rules (1 relaxation, 1 complementary). The hand-engineered neuro-symbolic baseline used as the primary comparison in the main paper.
- **LLM-Flax:** GNN replaced by LLM zero-shot scoring; manual rules replaced by LLM-generated rules. The proposed fully automated system.

Key takeaways Three findings stand out.

First, *Pure FD is a surprisingly strong baseline on SR alone*: it achieves SR 1.000 on all benchmarks up to 15×15 Medium, but at the cost of dramatically higher planning time (6–27 s) compared to rule-guided systems (1–9 s). For real-time deployment, this time gap is often prohibitive.

Second, *PLOI (GNN without rules) is frequently the weakest configuration*, falling below even Pure FD on several benchmarks (e.g., 12×12 Hard: SR 0.580 vs. 1.000). This reveals that the GNN score alone, without relaxation and complementary rules to structure the pruned problem, can lead to invalid or incomplete planning sets that prevent any solution. *The rules are the critical component*, not the scoring model.

Third, *the LLM-generated rules in LLM-Flax capture the essential benefit of manual rules* across the benchmarks where rules matter, while eliminating the rule-engineering cost entirely. At 15×15 Hard, LLM-Flax reaches SR 1.000 where Pure FD (0.333) and PLOI (0.700) both struggle. At 12×12 Expert, the gap between rule-guided and rule-free configurations is smaller than at 15×15 Hard but the ranking is the same (LLM-Flax 0.700 > Manual 0.667 > PLOI 0.433 > Pure FD 0.333), consistent with rules mattering more as problem

TABLE X: Full four-way comparison across all eight MazeN-amo benchmarks. SR = success rate ($\uparrow$). Pure FD and PLOI results from Appendix experiments; Manual and LLM-Flax from Table I.

Task	Config	SR	Time (s)	Len
10×10 Easy	Pure FD	1.000	0.671	15.4
	PLOI	0.900	0.761	17.6
	Manual	1.000	0.932	17.4
	LLM-Flax	1.000	0.898	17.1
10×10 Medium	Pure FD	1.000	1.400	15.7
	PLOI	0.920	0.840	16.5
	Manual	0.960	1.997	19.8
	LLM-Flax	1.000	0.949	16.6
10×10 Hard	Pure FD	1.000	3.030	19.2
	PLOI	0.820	1.310	19.2
	Manual	0.960	2.200	19.8
	LLM-Flax	0.960	1.600	19.3
12×12 Medium	Pure FD	1.000	6.190	21.1
	PLOI	0.900	1.320	21.8
	Manual	0.960	1.914	22.1
	LLM-Flax	0.980	2.151	22.3
12×12 Hard	Pure FD	1.000	17.59	28.8
	PLOI	0.580	1.270	26.3
	Manual	0.880	3.965	29.8
	LLM-Flax	0.920	4.529	29.2
12×12 Expert	Pure FD	0.333	26.19	29.7
	PLOI	0.433	1.090	26.1
	Manual	0.667	4.528	31.5
	LLM-Flax	0.700	5.982	29.3
15×15 Medium	Pure FD	1.000	27.33	31.8
	PLOI	0.767	5.090	30.0
	Manual	0.967	8.740	30.9
	LLM-Flax	0.967	9.007	31.0
15×15 Hard	Pure FD	0.333	34.37	29.3
	PLOI	0.700	3.750	32.0
	Manual	0.900	11.901	36.2
	LLM-Flax	1.000	10.979	36.0

structure gets harder to search blindly.

For each of the eight benchmarks we show three views of a representative solved problem: (a) the full problem with all objects; (b) LLM object importance scores assigned zero-shot (Stage 3); (c) the pruned object set after thresholding and the final plan found by LLM-Flax. The left block covers the four smaller grids (10×10 and 12×12 Medium); the right block covers the four harder grids (12×12 Hard/Expert and 15×15).

Fig. 6 shows MazeNamo; Fig. 7 shows the same three-view trace for a solved instance of each of the two further domains introduced in Section V-G, using LLM-Flax (Gemma3-

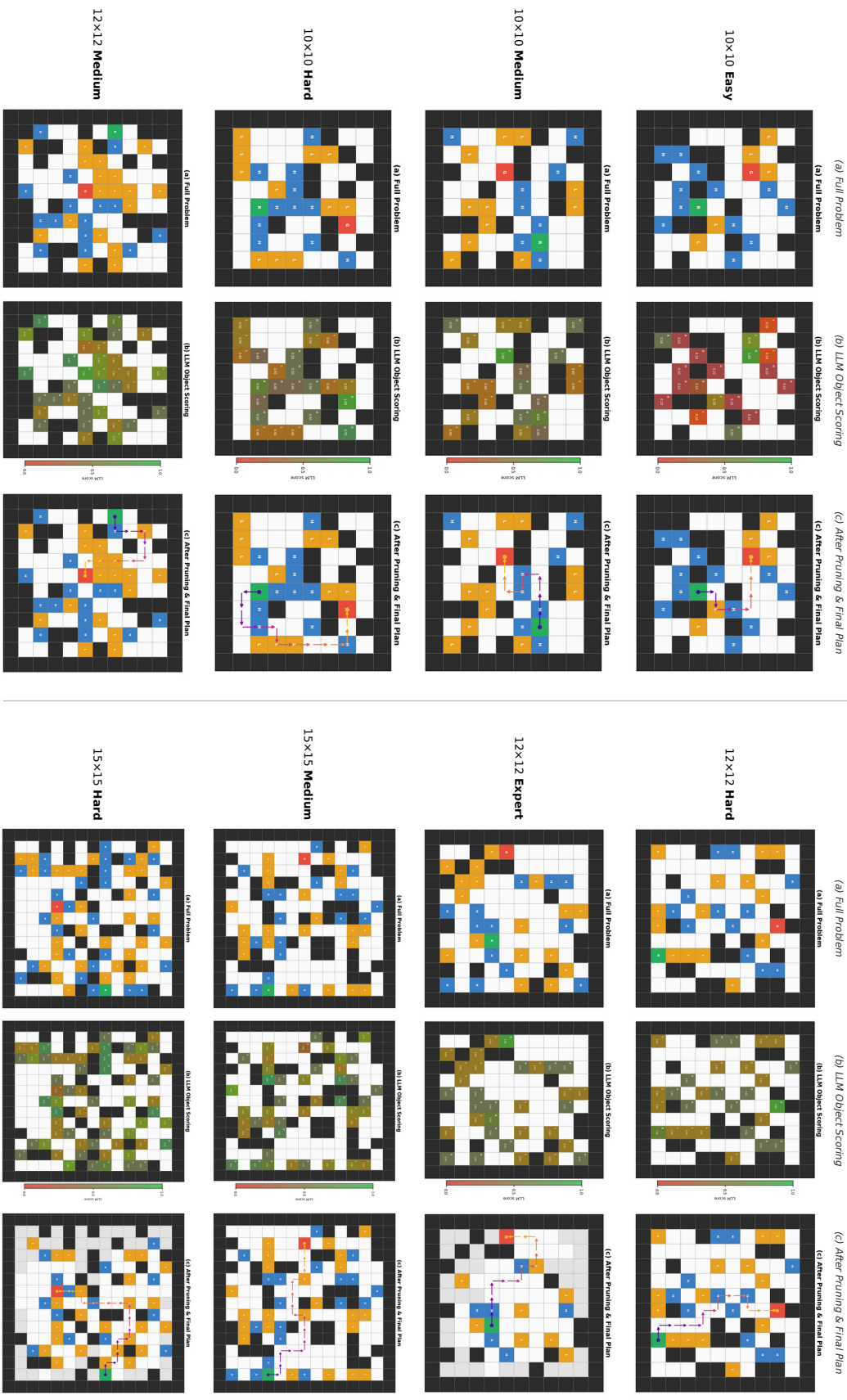


Fig. 6: Qualitative planning traces for all eight benchmarks. Each row is one benchmark (label at left); columns show (a) the full problem, (b) LLM object importance scores (Stage 3, zero-shot), and (c) the pruned object set and final plan produced by LLM-Flax.

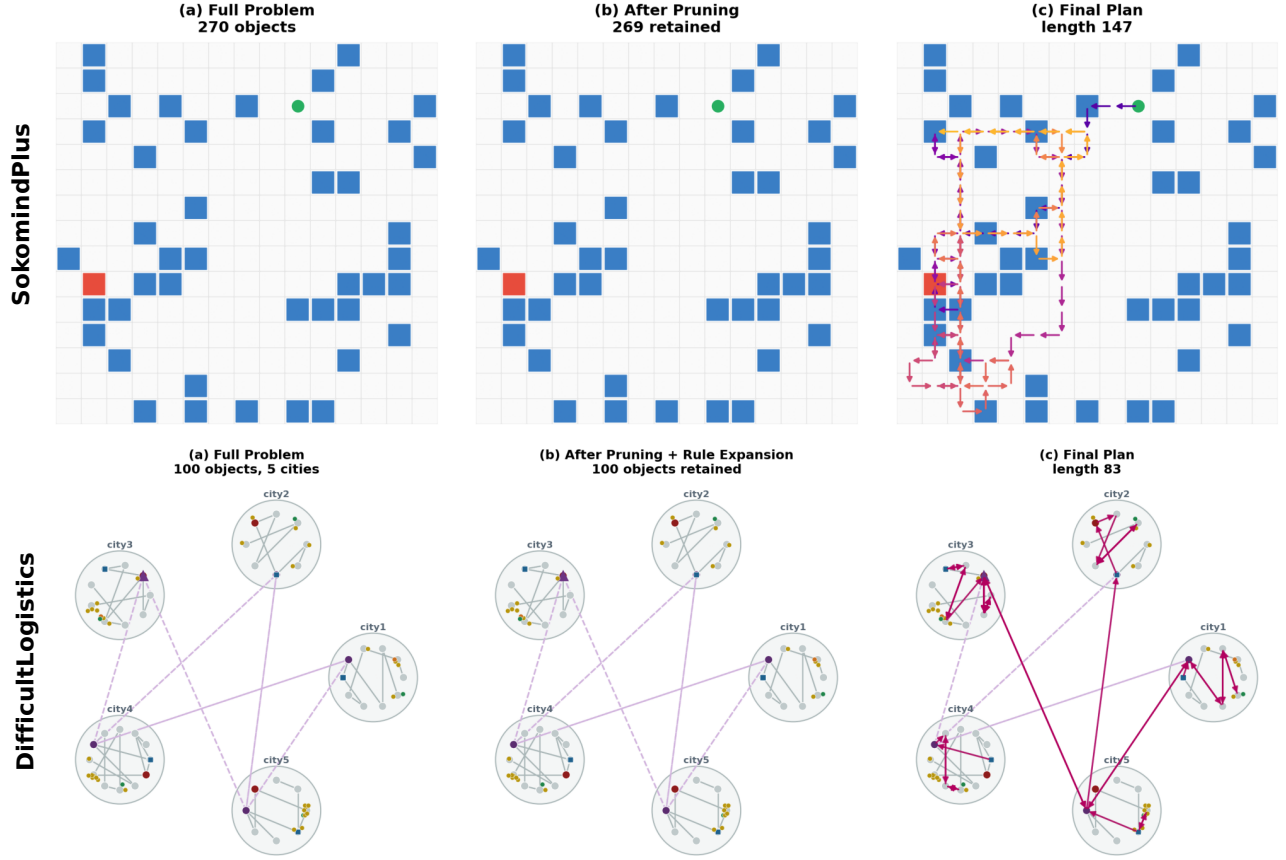


Fig. 7: Qualitative planning traces on the two further domains from Section V-G (LLM-Flax, Gemma3-12B rules, no-guidance scorer). **Top (SokomindPlus)**: grid view, same convention as Fig. 6 (blue = box, red = goal box, green = robot); column (c) traces the solved robot path. **Bottom (DifficultLogistics)**: city-clustered graph view (locations on inner circles grouped by city, cities on an outer ring, dashed lines = air-links); column (c) traces truck/airplane routes in the found plan. Column (b) here shows all 100 objects retained – this instance is the relaxation-fallback (Section V-G) actually engaging: Step 2 relaxation was attempted and found unsolvable, so the pipeline fell back to the unrelaxed object set before finding the plan in (c).

12B rules) with the no-guidance scorer. SokomindPlus (grid, top row) uses the same MazeNamo-style panel as Fig. 6; DifficultLogistics (bottom row) is not grid-based, so panels instead show a city-clustered graph (locations within a city on an inner circle, cities arranged on an outer circle, dashed edges are air-links). Column (b) for DifficultLogistics shows 100/100 objects retained: this is the relaxation-fallback path (Section V-G) actually firing on this instance—Step 2 relaxation was attempted, found unsolvable, and the pipeline fell back to the full object set for the final plan shown in (c).